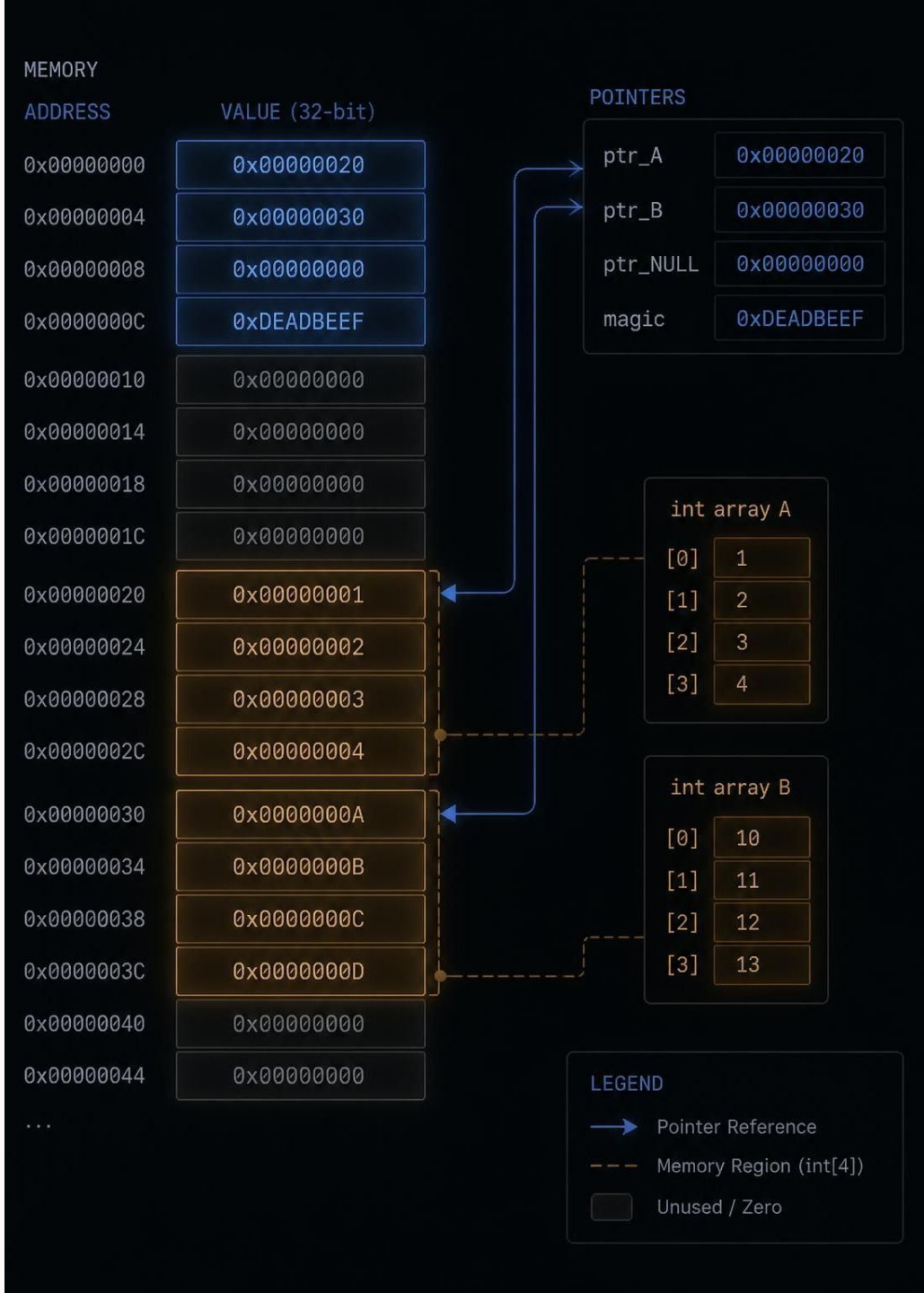
Puntatori e Array in C

Aritmetica dei puntatori, array come puntatori, stringhe e puntatori

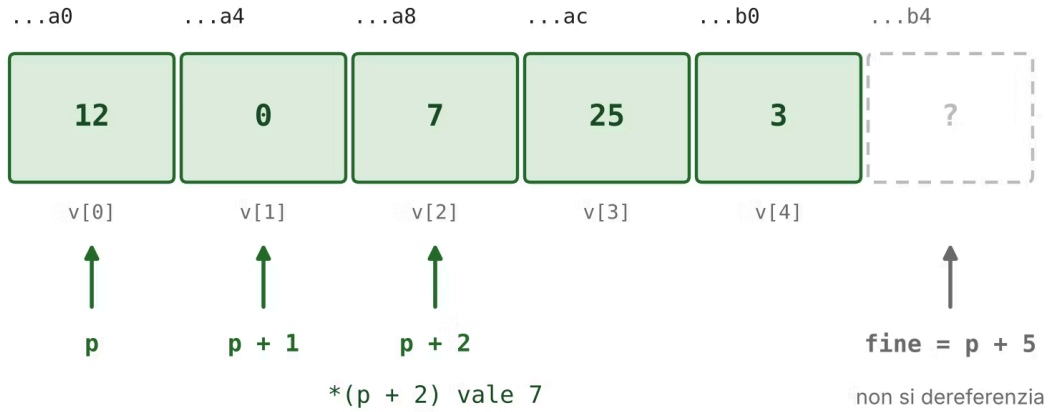
LINGUAGGIO C | PUNTATORI | ARRAY



p + i: Spostarsi in un Array

Se p punta a un elemento di un array, p + i punta all'elemento che si trova i posizioni più avanti. L'indirizzo non cresce di i byte ma di i × sizeof(*p) byte: il compilatore conosce il tipo puntato.

Le parentesi sono necessarie: *(p + 2) è l'elemento di indice 2, *p + 2 è il valore di v[0] più 2.



Ogni passo p + 1 avanza di sizeof(int) = 4 byte: ...a0, ...a4, ...a8 (indirizzi esadecimali abbreviati).

```
int v[5] = {12, 0, 7, 25, 3};
int *p = &v[0];
*(p + 2) = 9;       /* v[2] vale 9 */
```

Scorrere un Array con p++

Il problema: elaborare tutti gli elementi di un array senza usare un indice.

La soluzione: un puntatore che avanza con p++ e un secondo puntatore, fine, che contiene l'indirizzo subito dopo l'ultimo elemento.

Operazione	Effetto
p++	elemento successivo
p--	elemento precedente
p < fine	ci sono ancora elementi

fine si calcola e si confronta, ma non si dereferenzia mai: dopo l'ultimo elemento non c'è nulla che appartenga all'array.

```
const int *fine = &passi[0] + GIORNI;
const int *p;

for (p = &passi[0]; p < fine; p++) {
   totale = totale + *p;
}
```

Differenza e Confronto tra Puntatori

Due puntatori allo stesso array si possono sottrarre e confrontare.

Espressione	Significato
q - p	quanti elementi separano p da q (non quanti byte)
p - &v[0]	l'indice dell'elemento puntato
p < q	p viene prima di q
p == q	stesso elemento

Esempio: sul profilo di un sentiero, cima - inizio è il chilometro della cima e oltre_bosco < cima dice se si esce dal bosco prima della cima.

```
cima = inizio;
for (p = inizio; p < fine; p++) {
    if (*p > *cima) {
        cima = p;
    }
}
km = cima - inizio;       /* km della cima */
```

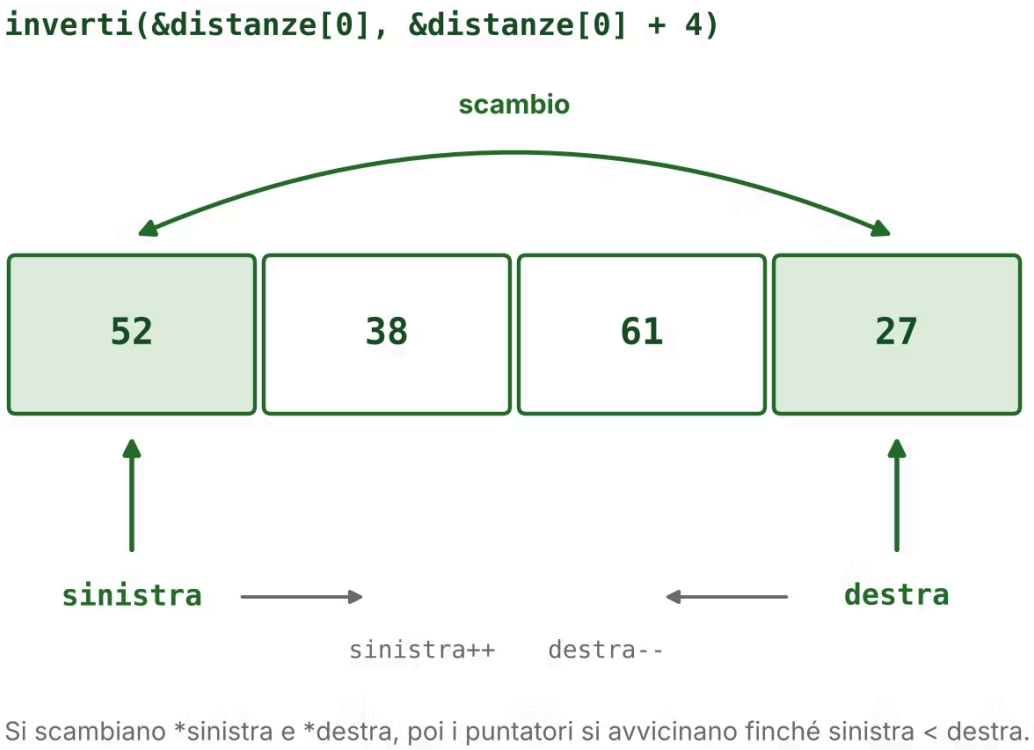
Due Puntatori sullo Stesso Array

Il problema: invertire l'ordine delle tappe di una gita per preparare il percorso di ritorno, senza un secondo array.

La soluzione: due puntatori partono dagli estremi, scambiano i valori e si avvicinano finché sinistra < destra.

Con più puntatori si risolvono anche la media mobile (un puntatore alla finestra di 3 letture) e la fusione di due classifiche ordinate (pa, pb, pc).

```
while (sinistra < destra) {
    scambio = *sinistra;
    *sinistra = *destra;
    *destra = scambio;
    sinistra++;
    destra--;
}
```



Si scambiano *sinistra e *destra, poi i puntatori si avvicinano finché sinistra < destra.

Il Nome dell'Array è un Indirizzo

In quasi tutte le espressioni il nome di un array diventa l'indirizzo del primo elemento: indici e puntatori sono due modi di scrivere la stessa cosa.

Con gli indici	Con i puntatori
&v[0]	v
&v[i]	v + i
v[i]	*(v + i)

	int v[5]	int *p = v
sizeof	20 byte: tutto l'array	8 byte: un indirizzo
si può spostare?	no: v++ è un errore	sì: p++

Dopo p = v si può scrivere anche p[i]: la scrittura con le parentesi quadre funziona con qualsiasi puntatore.

Array come Parametri

Il problema: una funzione che riceve un array non conosce la sua dimensione e sembra lavorare su una copia.

La soluzione: alla funzione arriva solo l'indirizzo del primo elemento. Come parametro int v[] e int *v sono equivalenti; il numero di elementi va passato a parte.

Dentro la funzione sizeof(v) dà 8, la dimensione di un indirizzo: il conto sizeof(v) / sizeof(v[0]) funziona solo dove l'array è dichiarato.

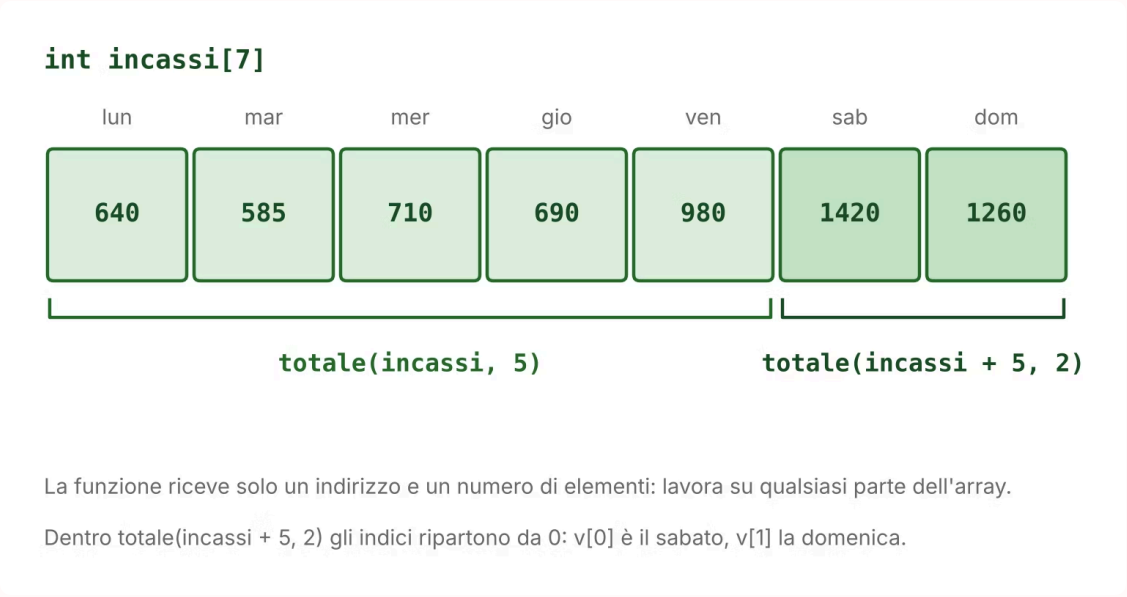
```
int totale(const int v[], int n);
int massimo(const int *v, int n);
int correggi_mancanti(int *v, int n);  /* modifica l'array del chiamante */
```

Passare Solo una Parte dell'Array

Il problema: confrontare gli incassi dei giorni feriali con quelli del fine settimana usando le stesse funzioni.

La soluzione: si passa l'indirizzo da cui partire e il numero di elementi. Dentro la funzione gli indici ripartono da 0.

Una posizione restituita dalla funzione è relativa al sottoarray: il chiamante aggiunge lo scostamento (5 + pos).



La funzione riceve solo un indirizzo e un numero di elementi: lavora su qualsiasi parte dell'array.

Dentro totale(incassi + 5, 2) gli indici ripartono da 0: v[0] è il sabato, v[1] la domenica.

```
totale(incassi, 5);         /* lunedì-venerdì */
totale(incassi + 5, 2);     /* sabato e domenica */
occupa(posti + i, posti + i + k);   /* intervallo [inizio, fine) */
```

Scorrere una Stringa

Una stringa è un array di char terminato da '\0'. Si scorre con un puntatore che avanza finché il carattere puntato non è il terminatore.

Ogni char occupa 1 byte: gli indirizzi dei caratteri crescono di 1.

```
const char *p = citta;

while (*p != '\0') {
    p++;
}
lunghezza = p - citta;      /* 6 */
printf("%s", citta + 2);    /* rino */
```



Un char occupa 1 byte: gli indirizzi (esadecimali) crescono di 1, da ...f0 a ...f6.

Array di char e Letterali

	char giorno[] = "lunedi";	**const char *stato = "APERTO";**
che cos'è	un array con una copia del testo	un puntatore a un testo costante
sizeof	7 (6 lettere + '\0')	8 (un indirizzo)
modificare i caratteri	sì: giorno[0] = 'L'	no: sola lettura
cambiare testo	solo copiando	sì: stato = "CHIUSO";

Un puntatore a un letterale si dichiara sempre const char *: un tentativo di modifica diventa un errore di compilazione invece di un'interruzione del programma.

```
if (ora >= APERTURA && ora < CHIUSURA) {
    stato = "APERTO";
} else {
    stato = "CHIUSO";
}
```

Il Puntatore di strchr e strstr

Nel capitolo 4 il risultato di strchr e strstr si confrontava solo con NULL. Ora il puntatore restituito si usa davvero.

printf("%.*s", n, p) stampa al massimo n caratteri a partire da p: la parola non si copia, basta sapere dove inizia e quanto è lunga.

```
const char *chiocciola = strchr(email, '@');

if (chiocciola != NULL) {
    pos = chiocciola - email;          /* 12 */
    printf("%s", chiocciola + 1);       /* scuola.it */
    printf("%.*s", pos, email);         /* giulia.rossi */
}

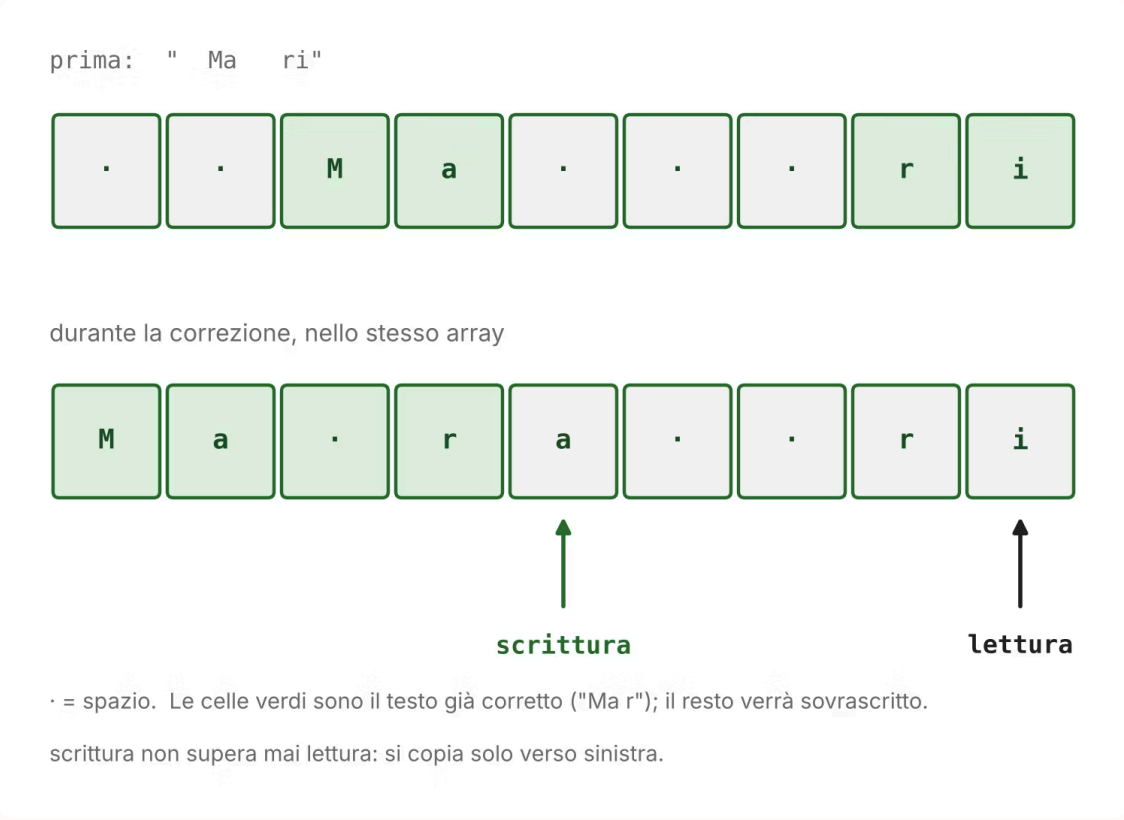
char *a_capo = strchr(riga, '\n');      /* dopo fgets */
if (a_capo != NULL) {
    *a_capo = '\0';
}
```

Lettura e Scrittura sulla Stessa Stringa

Il problema: togliere gli spazi in eccesso da un testo senza usare un secondo array.

La soluzione: un puntatore di lettura esamina tutti i caratteri, un puntatore di scrittura indica dove copiare il prossimo carattere da tenere.

scrittura non supera mai lettura: si copia solo verso sinistra e non si sovrascrive nulla che debba ancora essere letto. Alla fine si scrive '\0' in *scrittura.



Errori Comuni da Evitare

1

Scrivere *p + 2 invece di *(p + 2).

2

Dereferenziare fine, l'indirizzo dopo l'ultimo elemento.

3

Far uscire dall'array un gruppo di elementi (la finestra si ferma a fine - 3).

4

Calcolare la dimensione dell'array con sizeof dentro la funzione.

5

Tentare v++ sul nome di un array.

6

Dimenticare lo scostamento di una posizione restituita su un sottoarray.

7

Modificare un letterale o usare strchr senza controllare NULL.

8

Dimenticare il '\0' finale quando si copia una stringa con i puntatori.

Riepilogo Concetti

Operazione	Tecnica	Note
Elemento i posizioni dopo	*(p + i)	avanza di i × sizeof(*p) byte
Scorrere un array	for (p = &v[0]; p < fine; p++)	fine = &v[0] + N
Indice di un elemento	p - &v[0]	differenza tra puntatori
Posizione relativa	p < q, p == q	stesso array
Inversione, fusione	due o tre puntatori	sinistra / destra, pa / pb / pc
Nome dell'array	v == &v[0]	non si sposta
Array come parametro	const int *v, int n	n passato a parte
Parte di un array	f(v + k, n)	indici da 0 nella funzione
Scorrere una stringa	while (*p != '\0')	lunghezza p - s
Testo costante	const char *s = "..."	sola lettura
Risultato di strchr	p - s, p + 1	dopo il controllo NULL
Correzione sul posto	lettura e scrittura	scrittura ≤ lettura

SEZIONE 1

Esempi Pratici: Aritmetica dei Puntatori

Concetti base — tre file su p + i, p++ e differenza tra puntatori.

1 – 010-somma-di-un-intero.c

Problema: posti liberi e tariffe ai piani di un parcheggio.

Tecnica: p + i e *(p + i) con array di int e di double.

2 – 020-incremento-e-scorrimento.c

Problema: passi giornalieri di un contapassi.

Tecnica: p++ fino a fine, all'indietro con p--.

3 – 030-differenza-e-confronto.c

Problema: profilo di un sentiero di montagna.

Tecnica: cima - inizio, oltre_bosco < cima, cima == fine - 1.

SEZIONE 1

Esempi Pratici: Problemi di Aritmetica dei Puntatori

040 – Media mobile della CO2

Puntatore alla finestra di 3 letture: quando aprire le finestre dell'aula.

050 – Ora di punta

Consumi elettrici della scuola: puntatore al massimo e fascia di alto consumo.

060 – Percorso di ritorno

Gita in bicicletta: inversione dei tratti con due puntatori agli estremi.

070 – Serie di vittorie

Puntatore all'inizio della serie in corso, serie più lunga del campionato.

080 – Classifica unita

Due batterie di 100 metri fuse in una classifica con tre puntatori.

SEZIONE 2

Esempi Pratici: Array come Puntatori

Concetti base — tre file sul nome dell'array e sui parametri.

1 – 010-nome-array-e-indirizzo.c

Problema: posti liberi nelle aule studio.

Tecnica: v e &v[0], quattro modi di leggere un elemento, sizeof.

2 – 020-parametri-array-e-puntatori.c

Problema: pioggia di una settimana con misure mancanti.

Tecnica: int v[] e int *v come parametri, modifica dell'array.

3 – 030-sottoarray.c

Problema: ingressi mensili in palestra.

Tecnica: somma(v + 3, 3) e intervalli [inizio, fine).

SEZIONE 2

Esempi Pratici: Problemi su Array e Puntatori

040 – Vendite settimanali

Le stesse funzioni su feriali (incassi, 5) e fine settimana (incassi + 5, 2).

050 – Posti sul treno

Ricerca di k posti liberi vicini e prenotazione di un intervallo.

060 – Primo giorno di gelo

Ricerca su un periodo scelto, passato come sottoarray.

070 – Backup del registro

Copia di un blocco con due puntatori, confronto e ripristino dei voti.

080 – Istogramma dei voti

Array dei conteggi riempito da una funzione tramite int *.

SEZIONE 3

Esempi Pratici: Stringhe e Puntatori

Concetti base — tre file su stringhe, letterali e funzioni.

1 – 010-scorrere-una-stringa.c

Problema: annuncio del tabellone di una stazione.

Tecnica: while (*p != '\0'), lunghezza p - s, stampa al contrario.

2 – 020-letterali-e-array-di-char.c

Problema: cartello APERTO / CHIUSO di un negozio.

Tecnica: char s[] e const char *s, scelta di un letterale.

3 – 030-funzioni-con-puntatori.c

Problema: indirizzo e-mail di uno studente.

Tecnica: lunghezza, copia e confronto con i puntatori; puntatore di strchr.

SEZIONE 3

Esempi Pratici: Problemi su Stringhe e Puntatori

040 – Sigle

Iniziali delle parole lunghe: Istituto Tecnico Industriale Statale diventa ITIS.

050 – Spazi in eccesso

Normalizzazione dei testi di un modulo con lettura e scrittura.

060 – Campi CSV

Scomposizione di righe cognome;nome;classe;voto e media dei voti.

070 – Parola più lunga

Titoli del giornalino: parola più lunga stampata con %.*s.

080 – Filtro della chat

Parole vietate trovate con strstr e sostituite con asterischi.